1. Introduction. This is the GWEB version of the classic `wc` word-counting program, modeled on the example that accompanies CWEB. It reads each file named on the command line (or standard input when no files are given) and reports the number of lines, words, and characters it contains, followed by a grand total when more than one file is processed.

The program illustrates the main ingredients of a literate program: starred and ordinary sections, prose interleaved with code, and named sections (here called *refinements*) that let us present the code in the order that best explains it rather than the order the compiler needs.

2. The whole program is a single Go file in package *main*. Its skeleton names three refinements that are filled in below; *gtangle* assembles them into the order Go expects, while *gweave* typesets them in the order shown here.

```
package main
import (
    "bufio"
    "fmt"
    "io"
    "os"
)
⟨ Type definitions 3 ⟩
⟨ Subroutines 4 ⟩
func main() {
    ⟨ Count each file and print a grand total 5 ⟩
}
```

3. Counting. For every input we accumulate three integers, kept together in one struct. The format directive on the next line asks *gweave* to typeset the type **counts** in bold, the way it sets predeclared types such as **int**.

⟨ Type definitions 3 ⟩ ≡

```
type counts struct {
    lines, words, chars int
}
```

This code is used in section 2.

4. It is handy to fold one file's tally into a running total.

⟨ Subroutines 4 ⟩ ≡

```
func (c *counts) add(d counts) {
    c.lines += d.lines
    c.words += d.words
    c.chars += d.chars
}
```

This code is also defined in sections 8, 10.

This code is used in section 2.

5. The driver decides between filter mode and file mode, then prints a total.

⟨ Count each file and print a grand total 5 ⟩ ≡

```
files := os.Args[1:]
var total counts
⟨ Count the standard input if no files were given 6 ⟩
for _, name := range files {
    ⟨ Count one named file 7 ⟩
}
if len(files) > 1 {
    report(total, "total")
}
```

This code is used in section 2.

6. With no arguments we behave like a Unix filter, reading standard input.

⟨ Count the standard input if no files were given 6 ⟩ ≡

```
if len(files) == 0 {
    report(countReader(os.Stdin), "")
    return
}
```

This code is used in section 5.

7. Otherwise each named file is opened, counted, closed, and added to the total. A file that cannot be opened produces a diagnostic but does not stop the run.

⟨ Count one named file 7 ⟩ ≡

```
f, err := os.Open(name)
if err ≠ nil {
    fmt.Fprintln(os.Stderr, "wc:", err)
    continue
}
c := countReader(f)
f.Close()
report(c, name)
total.add(c)
```

This code is used in section 5.

8. The scanner. The heart of the program reads one *io.Reader* byte by byte. A *word* is a maximal run of characters that are not white space, so we remember whether we are currently inside a word.

⟨ Subroutines 4 ⟩ +≡

```
func countReader(r io.Reader) counts {
    var c counts
    in := bufio.NewReader(r)
    inWord := false
    for {
        b, err := in.ReadByte()
        if err ≠ nil {
            break
        }
        c.chars++
        ⟨ Update the counts for byte b 9 ⟩
    }
    return c
}
```

9. Every byte is a character; a newline additionally ends a line; and any white space ends the current word, while the first non-space after a gap starts a new one.

⟨ Update the counts for byte b 9 ⟩ ≡

```
if b == '\n' {
    c.lines++
}
switch b {
case ' ', '\t', '\n', '\r', '\f', '\v':
    inWord = false
default:
    if !inWord {
        inWord = true
        c.words++
    }
}
```

This code is used in section 8.

10. Output. Finally, one helper prints a single row of the report. The grand-total row and the standard-input row are distinguished only by their trailing label.

⟨ Subroutines 4 ⟩ +≡

```
func report(c counts, name string) {
    if name == "" {
        fmt.Printf("%8d %8d %8d\n", c.lines, c.words, c.chars)
    } else {
        fmt.Printf("%8d %8d %8d %s\n", c.lines, c.words, c.chars, name)
    }
}
```

11. Index. Names and refinements are cross-referenced automatically below.

⟨ Count each file and print a grand total 5 ⟩ Used in section 2
⟨ Count one named file 7 ⟩ Used in section 5
⟨ Count the standard input if no files were given 6 ⟩ Used in section 5
⟨ Subroutines 4 ⟩ Used in section 2
⟨ Type definitions 3 ⟩ Used in section 2
⟨ Update the counts for byte b 9 ⟩ Used in section 8

WC